

Campionamento statistico

Un "aneddoto" storico per capire idea dell'importanza del campionamento statistico: La storia dei *voti di paglia* (straw polls)

Il campionamento statistico, cioè l'idea di stimare le caratteristiche di una popolazione osservandone solo una parte, ha radici antiche.

Un esempio storico per dare l'idea è quello relativo all'uso dei cosiddetti **straw polls**, letteralmente "voti di paglia" negli Stati Uniti .

Cosa sono i "voti di paglia"?

Il termine *straw poll* deriva dalla metafora del filo di paglia che lasciato nel vento indica la direzione del vento.

Per uscire dalla metafora, l'idea di fondo è che un piccolo sottoinsieme di persone (un campione) può indicare la tendenza dell'intera popolazione (così come un filo di paglia la direzione del vento).

I primi straw polls nella storia americana

I primi esempi organizzati risalgono agli anni **1820–1840** negli Stati Uniti, quando giornali e comunità locali iniziarono a raccogliere opinioni politiche tra piccoli gruppi di cittadini.

Il *Harrisburg Pennsylvanian* condusse un piccolo sondaggio per prevedere l'esito delle elezioni presidenziali tra Andrew Jackson e John Quincy Adams. Sorprendentemente, il sondaggio indovinò il vincitore in quella città (Jackson), anche se non il risultato nazionale.

Da allora la pratica iniziò a diffondersi.

Il fenomeno si diffuse nel tempo a varie riviste. Per esempio alla rivista **The Literary Digest** che, dagli anni 1890 fino al 1936, effettuò enormi sondaggi elettorali.

I loro campioni, però, non erano "scientifici" perché venivano inviati milioni di questionari a automobilisti, abbonati a elenchi telefonici, lettori della rivista.

Questi sondaggi erano grandi, ma **distorti** perché rappresentavano soprattutto una fascia particolare della popolazione: il ceto medio/benestante (chi all'epoca possedeva una macchina o un telefono e era abbonato a riviste).

Literary Digest vs. George Gallup

Nel 1936 The Literary Digest predisse la vittoria del repubblicano Alf Landon con 57% dei voti. Lo statistico George Gallup, invece, utilizzando un piccolo campione basato su quote demografiche predisse correttamente la netta vittoria di Franklin D. Roosevelt.

La previsione errata della rivista, pur essendo basata su un numero di acquisizioni molto più elevato, dimostrò che:

Non conta la dimensione del campione, ma come viene selezionato.

Anche questo evento contribuì a far nascere la statistica moderna applicata ai sondaggi.

L'approccio statistico moderno si sviluppa nel tempo:

- si formalizzano tecniche di campionamento casuale,
 - nascono istituti demoscopici scientifici,
 - si diffondono metodi come campionamento casuale semplice, stratificato, a cluster.
-

▼ Il Campionamento Statistico

Il **campionamento statistico** è l'insieme delle tecniche utilizzate per selezionare una parte (campione) di una popolazione al fine di fare inferenze sulla popolazione stessa. È uno dei concetti fondamentali della statistica inferenziale.

Obiettivo del campionamento

Poiché spesso è impossibile osservare o misurare un'intera popolazione, si preleva un **campione rappresentativo**. Lo scopo è stimare parametri della popolazione, come:

- media
- proporzione
- varianza
- distribuzione
- correlazioni

Perché il campione sia utile, deve essere **casuale e rappresentativo**.

Concetti chiave

Popolazione

L'insieme completo degli individui/oggetti di interesse.

Campione

Sottoinsieme della popolazione scelto per l'analisi.

Parametro

Valore vero della popolazione (es. media μ). Non conosciuto.

Statistica

Valore calcolato dal campione (es. media campionaria $\bar{x}$). Serve a stimare il parametro.

Perché serve campionare?

- Riduce costi e tempi
 - Evita l'impossibilità pratica di misurare tutta la popolazione
 - Permette stime accurate se il campione è ben progettato
 - È fondamentale per sondaggi, studi clinici, controlli di qualità, apprendimento automatico
-

Tipi di campionamento

Campionamento casuale semplice (SRS)

Ogni individuo ha la **stessa probabilità** di essere scelto.

Esempio in Python che prende da df un campione di 100 dati:

```
df_sample = df.sample(n=100, random_state=42)
```

Vantaggi:

- facile da capire e implementare

Limiti:

- può essere costoso se la popolazione è molto grande
-

Campionamento sistematico

Si seleziona un'unità ogni k elementi.

Esempio: selezionare ogni 10° individuo.

Si usa quando la popolazione è ordinata.

Rischio: se esiste una periodicità nascosta, introduce bias.

Campionamento stratificato

La popolazione viene divisa in **strati omogenei** (es. sesso, età) e si estrae un campione da ciascuno strato.

Molto usato in:

- studi clinici
- sondaggi demografici

Vantaggi:

- garantisce rappresentatività anche per gruppi piccoli
-

Campionamento a cluster

Si dividono gli individui in **cluster** (gruppi naturali, es. scuole). Poi si selezionano interi cluster.

Vantaggi:

- economico e semplice in grandi popolazioni

Limiti:

- maggiore varianza rispetto allo stratificato

Campionamento multistadio

Combinazione di vari metodi (es. cluster + estrazione casuale dentro i cluster). Usato in sondaggi nazionali.

Cenni alla dimensione del campione

La dimensione ideale dipende da:

- livello di confidenza desiderato
- margine di errore accettabile
- variabilità della popolazione
- tipo di stima da fare

Esempio formula approssimata per una proporzione:

$$n = \frac{z^2 p(1-p)}{e^2}$$

dove:

- z = quantile normale (per 95% $\rightarrow 1.96$)
- p = proporzione attesa
- e = margine di errore

Errore campionario

L'**errore campionario** è la differenza tra il parametro vero e la statistica calcolata dal campione.

Non si può eliminare, ma si può:

- ridurre aumentando il campione
- controllare con metodi di campionamento corretti

Distribuzione campionaria

Se ripetessimo molte volte il campionamento e ogni volta calcolassimo una statistica (es. la media), otterremmo la **distribuzione campionaria**.

Teorema fondamentale che vedremo nella lezione successiva:

Teorema del Limite Centrale (CLT)

Per campioni sufficientemente grandi, la media campionaria segue una distribuzione normale, indipendentemente dalla distribuzione della popolazione.

Questo permette di:

- costruire intervalli di confidenza
- fare test statistici

Campionamento in Python: esempi semplici

Campionamento casuale semplice

```
sample = df.sample(frac=0.2, random_state=0)
```

Campionamento stratificato

```
from sklearn.model_selection import StratifiedShuffleSplit

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

for train_idx, test_idx in split.split(df, df["strato"]):
    train = df.iloc[train_idx]
    test = df.iloc[test_idx]
```

Campionamento con pesi

```
sample = df.sample(n=100, weights='peso', random_state=42)
```

Bias di campionamento

Errori comuni che rendono il campione **non rappresentativo**:

- campionamento di convenienza
- mancata risposta nei sondaggi
- selezione non casuale
- cluster troppo differenziati tra loro
- esclusione sistematica di un gruppo

Conseguenza: stime distorte e non generalizzabili.

Fai doppio clic (o premi Invio) per modificare

Metodi per Generare Numeri Pseudocasuali

A John von Neumann è attribuita la seguente citazione:

"Non c'è niente di più difficile che generare numeri casuali con un computer"

In questa sezione il problema di generare numeri (pseudo)casuali con un computer e un esempio semplice di generatore di numeri pseudocasuali.

La generazione di numeri pseudocasuali è un elemento fondamentale in probabilità, statistica computazionale, machine learning, simulazioni Monte Carlo e crittografia.

In questa lezione anche **come vengono generati** e come **come usarli in Python**.

Che cosa sono i numeri pseudocasuali?

Un **numero pseudocasuale** è un numero generato da un algoritmo deterministico, progettato per *sembrare* casuale. Sono "pseudo" perché:

- non provengono da fenomeni naturali
- sono riproducibili se si conosce il *seme* (seed)
- hanno proprietà statistiche simili a numeri casuali veri

Esempio:

```
import numpy as np
np.random.seed(123)
np.random.rand()
```

Ogni volta che esegui questo codice ottieni lo stesso numero.

Generatori di numeri pseudocasuali (PRNG)

I moderni linguaggi di programmazione hanno integrati dei PRNG (Pseudo Random Number Generators). Sono algoritmi matematici che, partendo da un valore iniziale (*seed*), generano una sequenza di numeri.

Linear Congruential Generator (LCG)

Uno dei metodi classici e più semplici è il Linear Congruential Generator:

$$X_{n+1} = (aX_n + c) \mod m$$

Dove:

- a = moltiplicatore
- c = incremento
- m = modulo
- X_0 = seed

Semplice e molto veloce, ma non molto robusto.

Cenni ad altri metodi: Mersenne Twister

È il PRNG usato storicamente da **NumPy**. Caratteristiche:

- periodo enorme ($(2^{19937} - 1)$)
- distribuzione uniforme di alta qualità
- molto veloce

Esempio in Python:

```
import numpy as np
np.random.default_rng()
```

PCG (Permuted Congruential Generator)

Il generatore *moderno* introdotto in NumPy 1.17 con `default_rng()`:

- distribuzioni migliori
- più veloce del Mersenne Twister
- migliore statistica dei bit

Esempio:

```
from numpy.random import default_rng
rng = default_rng(42)
rng.normal(size=5)
```

Metodi per passare da una distribuzione uniforme ad altre distribuzioni

Se abbiamo uno (o più) algoritmo che genera una numero in modo uniforme, come possiamo generare valori le altre distribuzioni?

Molti algoritmi non generano direttamente numeri per qualsiasi distribuzione, ma **partono da numeri uniformi** e li trasformano tramite la cosiddetta **Inverse Transform Sampling**

Metodo che parte da una distribuzione uniforme generato, per esempio, come spiegato in precedenza.

1. Genera $U \sim \text{Uniform}(0, 1)$
2. Applica l'inversa della CDF:

$$X = F^{-1}(U)$$

Esempio: generare una variabile esponenziale (per esercizio vedi con si può invertire esponenziale):

```
import numpy as np

u = np.random.rand()
x = -np.log(1 - u) / 2 # lambda = 2
```

Metodo di Box–Muller

Usato per generare valori **normali** a partire da uniformi.

Formula:

$$Z_1 = \sqrt{-2 \ln U_1} \cos(2\pi U_2)$$
$$Z_2 = \sqrt{-2 \ln U_1} \sin(2\pi U_2)$$

In NumPy è incorporato.

2.3. Rejection Sampling (Accettazione-Rifiuto)

Usato quando la distribuzione è complicata:

- genera candidati da distribuzione facile
- accetta/rifiuta in base a una regola

Usato anche in Metropolis–Hastings (vedremo probabilmente in futuro quando vedremo metodo Montecarlo).

Come generare numeri pseudocasuali in Python

Con `numpy.random` (modern version)

```
from numpy.random import default_rng
rng = default_rng(123)

x = rng.normal(loc=0, scale=1, size=1000)
```

Con `random` (libreria standard, qualità minore)

```
import random
random.seed(123)
random.random()
```

Con distribuzioni di `scipy.stats`

```
from scipy.stats import expon
expon.rvs(scale=2, size=1000, random_state=123)
```

4.1 Impostare il seed (riproducibilità)

```
rng = default_rng(42)
```

Se usando `random`:

```
import random
random.seed(42)
```

Se usando `scipy.stats`:

```
expon.rvs(size=100, random_state=42)
```

✓ Esempio distribuzione esponenziale

Ricordiamo idea di base:

1. Genera un numero casuale

$$U \sim \text{Uniforme}(0, 1)$$

2. Calcola:

$$X = F^{-1}(U)$$

3. Il valore (X) ha distribuzione con CDF (F)

Perché funziona (intuizione)

La CDF $F(x)$ rappresenta la probabilità che una variabile casuale sia **minore o uguale a** x .

- $F(x)$ cresce da 0 a 1
- Se scegli un valore casuale $U \in [0, 1]$
- e trovi il punto x tale che $F(x) = U$

stai “trasferendo” la casualità uniforme sulla forma della distribuzione desiderata.

Da qui il termine **transform** (trasformazione).

CDF dell'esponenziale

$$F(x) = 1 - e^{-\lambda x}, \quad x \geq 0$$

Inversa della CDF

$$F^{-1}(u) = -\frac{1}{\lambda} \ln(1 - u)$$

Algoritmo

1. Genera $U \sim \text{Uniforme}(0, 1)$

2. Calcola:

$$X = -\frac{1}{\lambda} \ln(1 - U)$$

Quando si può usare

Il metodo è applicabile quando:

- la **CDF è nota**
- la **CDF è invertibile in forma chiusa**

Distribuzioni tipiche:

- uniforme
- esponenziale
- Pareto
- Weibull (in molti casi)
- distribuzioni discrete semplici

Limiti del metodo

Non utilizzabile facilmente se:

- la CDF non ha un'inversa esplicita
- l'inversa è costosa da calcolare

Per questo, per distribuzioni come la **normale**, si usano metodi alternativi:

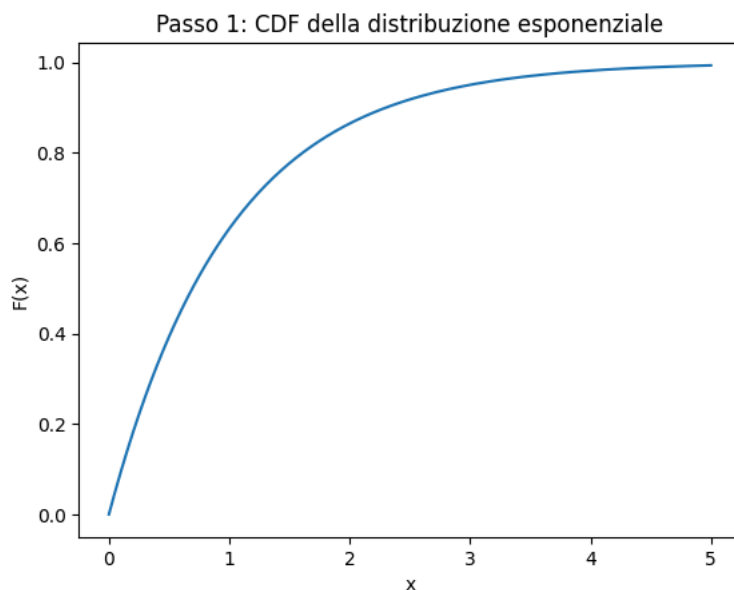
- Box-Muller
- Rejection sampling

```
# Esempio grafico CDF esponenziale per capire l'idea

import numpy as np
import matplotlib.pyplot as plt

lambda_ = 1.0
x = np.linspace(0, 5, 400)
cdf = 1 - np.exp(-lambda_ * x)

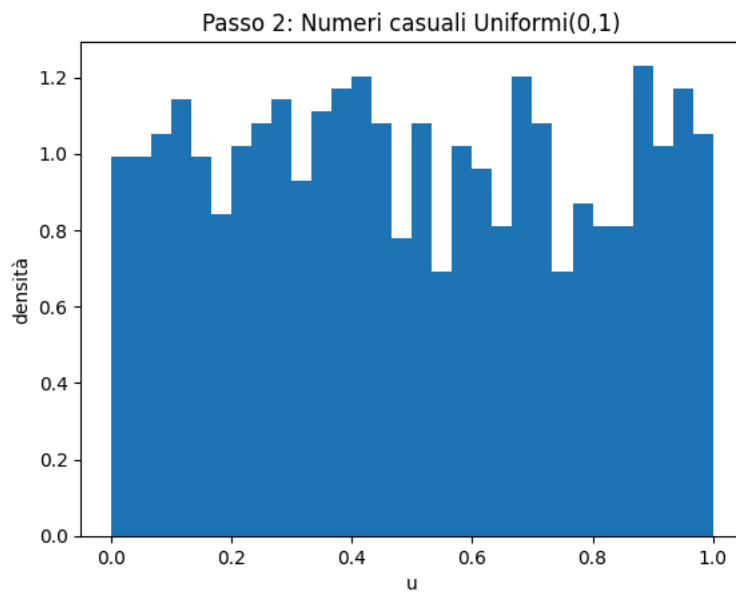
plt.figure()
plt.plot(x, cdf)
plt.xlabel("x")
plt.ylabel("F(x)")
plt.title("Passo 1: CDF della distribuzione esponenziale")
plt.show()
```



```
np.random.seed(0)
U = np.random.rand(1000)

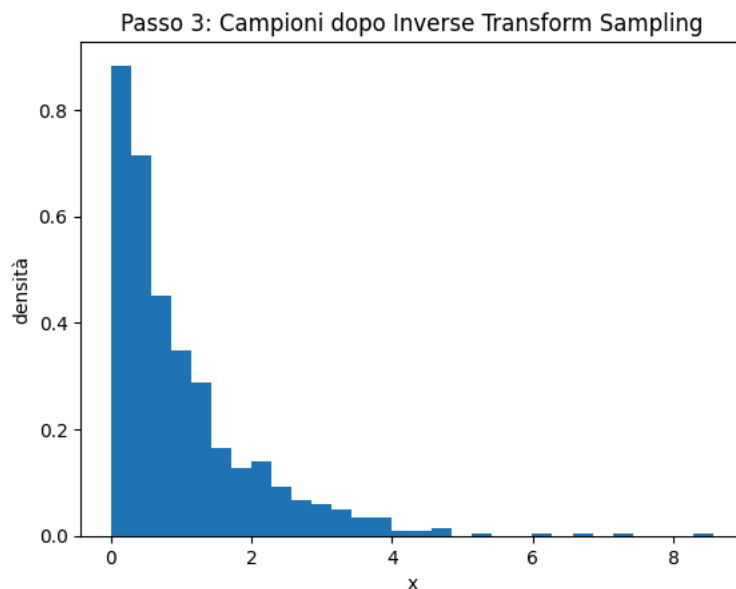
plt.figure()
```

```
plt.hist(U, bins=30, density=True)
plt.xlabel("u")
plt.ylabel("densità")
plt.title("Passo 2: Numeri casuali Uniformi(0,1)")
plt.show()
```



```
X = -np.log(1 - U) / lambda_

plt.figure()
plt.hist(X, bins=30, density=True)
plt.xlabel("x")
plt.ylabel("densità")
plt.title("Passo 3: Campioni dopo Inverse Transform Sampling")
plt.show()
```



Il **metodo di Box–Muller** è un algoritmo che permette di **generare numeri casuali con distribuzione normale (gaussiana)** a partire da numeri casuali **uniformi**. È molto importante in probabilità, statistica e simulazioni Monte Carlo, perché molti generatori di numeri casuali producono solo variabili uniformi.

▼ Obiettivo

Trasformare due variabili casuali indipendenti

$$U_1, U_2 \sim \text{Uniforme}(0, 1)$$

in due variabili casuali indipendenti

$$Z_1, Z_2 \sim \mathcal{N}(0, 1)$$

Formula del metodo di Box–Muller

Date U_1 e U_2 , si definiscono:

$$Z_1 = \sqrt{-2 \ln(U_1)} \cos(2\pi U_2) \quad Z_2 = \sqrt{-2 \ln(U_1)} \sin(2\pi U_2)$$

Le variabili (Z_1) e (Z_2) sono:

- **indipendenti**
- **distribuite normalmente**
- con **media 0** e **varianza 1**

Spiegazione

1. Consideriamo il piano cartesiano.
2. Il metodo genera un punto casuale nel piano usando:
 - un **raggio** $R = \sqrt{-2 \ln(U_1)}$
 - un **angolo** $\Theta = 2\pi U_2$
3. Il punto viene espresso in **coordinate polari**:

$$(R, \Theta)$$

4. Convertendo in coordinate cartesiane:

$$Z_1 = R \cos \Theta, \quad Z_2 = R \sin \Theta$$

Il risultato è una distribuzione **radialmente simmetrica**, tipica della normale bivariata.

Come funziona l'algoritmo?

- U_2 genera un angolo uniforme tra 0 e 2π
- U_1 controlla la distanza dal centro in modo che la densità dei punti decresca esponenzialmente
- Questo riproduce esattamente la densità della normale bivariata:

$$f(x, y) = \frac{1}{2\pi} e^{-\frac{x^2+y^2}{2}}$$

Vantaggi

- Metodo **esatto** (non approssimato)
- Produce **due gaussiane indipendenti** per ogni coppia di uniformi
- Ottimo per spiegare concetti teorici di probabilità

Svantaggi

- Computazionalmente più lento (logaritmi, seno e coseno)

Estensione a una normale generica

Per ottenere:

$$X \sim \mathcal{N}(\mu, \sigma^2)$$

si usa:

$$X = \mu + \sigma Z$$

dove (Z) è generato con Box–Muller.

```
import numpy as np
import matplotlib.pyplot as plt

# =====
# Metodo di Box-Muller
# =====

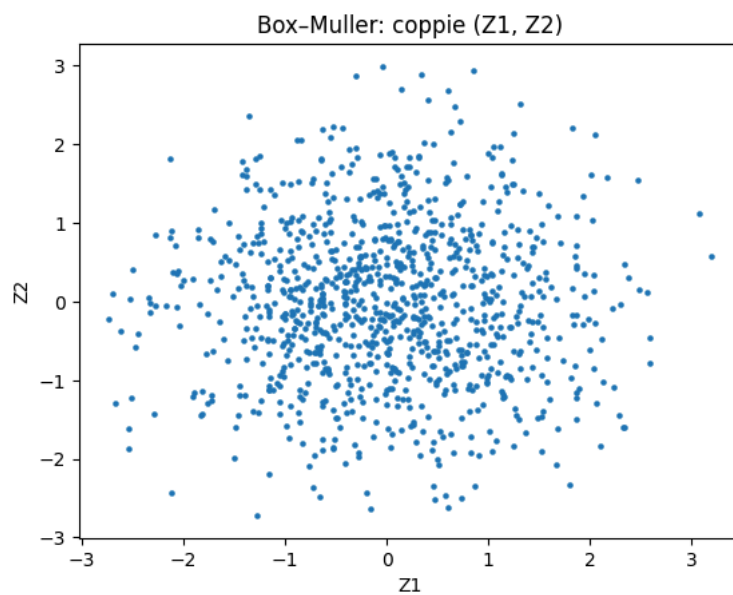
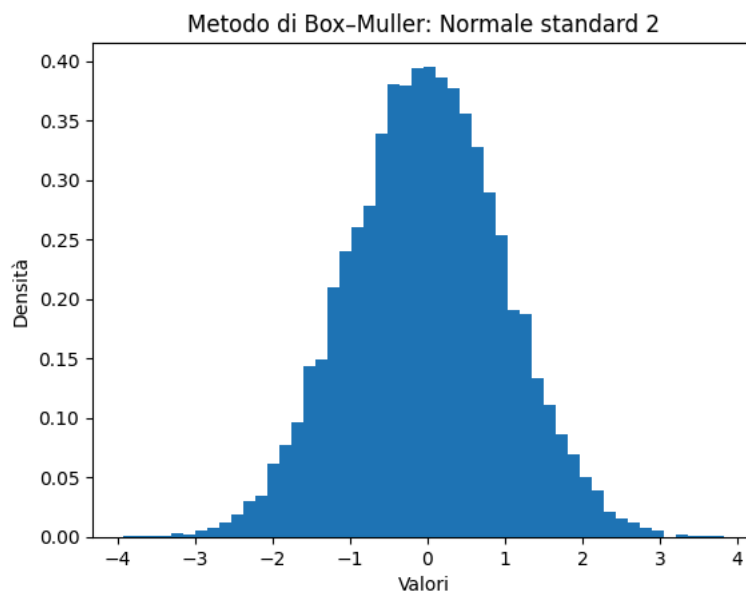
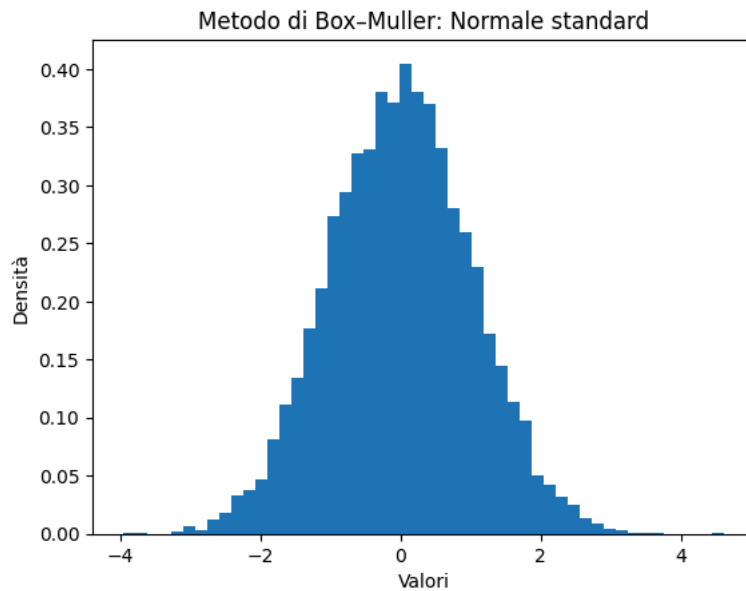
np.random.seed(42) # imposto il seme
n = 10000

# 1. Generazione di due Uniformi(0,1)
U1 = np.random.rand(n)
U2 = np.random.rand(n)
```

```
# 2. Trasformazione di Box-Muller
Z1 = np.sqrt(-2 * np.log(U1)) * np.cos(2 * np.pi * U2)
Z2 = np.sqrt(-2 * np.log(U1)) * np.sin(2 * np.pi * U2)

# =====
# Grafico 1: istogramma della Normale standard
# =====
plt.figure()
plt.hist(Z1, bins=50, density=True)
plt.xlabel("Valori")
plt.ylabel("Densità")
plt.title("Metodo di Box-Muller: Normale standard")
plt.show()

plt.figure()
plt.hist(Z2, bins=50, density=True)
plt.xlabel("Valori")
plt.ylabel("Densità")
plt.title("Metodo di Box-Muller: Normale standard 2")
plt.show()
# =====
# Grafico 2: scatter plot (indipendenza)
# =====
plt.figure()
plt.scatter(Z1[:1000], Z2[:1000], s=5)
plt.xlabel("Z1")
plt.ylabel("Z2")
plt.title("Box-Muller: coppie (Z1, Z2)")
plt.show()
```



✓ Esempio sui Linear Congruential Generators

L'equazione fondamentale che governa il **Linear Congruential Generator (LCG)** è una relazione di ricorrenza utilizzata per generare una sequenza di numeri pseudocasuali. L'equazione è la seguente:

$$X_{n+1} = (aX_n + c) \bmod m$$

dove:

- X_{n+1} : il numero successivo nella sequenza
- X_n : il numero corrente nella sequenza
- a : il **moltiplicatore**, una costante che determina la velocità di evoluzione della sequenza
- c : l'**incremento**, una costante aggiunta per modificare la sequenza
- m : il **modulo**, un intero positivo che specifica l'intervallo dei valori della sequenza

La formula dell'LCG prende il valore corrente X_n , applica il moltiplicatore a , aggiunge l'incremento c e infine applica l'operazione modulo m per produrre il valore successivo della sequenza, X_{n+1} .

La scelta di questi parametri è cruciale, poiché influenza le proprietà statistiche, il periodo e il comportamento della sequenza generata.

Il ruolo del seed

La sequenza inizia con un valore iniziale chiamato **seed**, indicato con X_0 . Il seed funge da punto di partenza per l'LCG e, se lo stesso seed viene utilizzato in esecuzioni multiple, l'LCG produrrà esattamente la stessa sequenza. Questo aspetto è fondamentale per la **riproducibilità** delle simulazioni.

Allo stesso tempo, evidenzia che gli LCG sono **pseudocasuali** e non veramente casuali, poiché l'intera sequenza è deterministica e dipende esclusivamente dai parametri scelti.

Parametri chiave: moltiplicatore, incremento, modulo e seed

Ogni parametro dell'equazione dell'LCG svolge un ruolo specifico nel modellare la sequenza di numeri generata. Comprenderli è essenziale per configurare un LCG efficace.

Moltiplicatore (a)

Il moltiplicatore deve essere scelto con attenzione per garantire un periodo lungo e buone proprietà di casualità. Valori comunemente usati derivano da studi teorici sul comportamento degli LCG, come quelli adottati in generatori storici (ad esempio quello del linguaggio C).

Incremento (c)

L'incremento viene aggiunto per evitare che la sequenza risulti troppo prevedibile. Se $c = 0$, l'LCG diventa un **generatore congruenziale moltiplicativo**, che presenta alcune limitazioni e può ridurre la qualità e l'intervallo dei numeri generati. Per gli LCG di uso generale, c viene solitamente scelto diverso da zero.

Modulo (m)

Il modulo determina l'intervallo dei valori generati. Nella maggior parte dei casi viene scelto come:

- una grande potenza di due (ad esempio 2^{32}) per efficienza computazionale, oppure
- un numero primo grande.

La scelta di m influisce fortemente sulla **periodicità** dell'LCG: valori più grandi consentono sequenze più lunghe prima che i numeri inizino a ripetersi.

Seed (X_0)

Il seed è il valore iniziale usato per inizializzare la sequenza. Seed diversi producono sequenze diverse, rendendolo uno strumento utile per generare più flussi di numeri casuali. È inoltre fondamentale per la riproducibilità: fissando il seed, è possibile ricreare esattamente la stessa sequenza, utile nel testing e nel debugging di modelli quantitativi.

(SOLO CENNI): Periodicità e proprietà statistiche degli LCG

La **periodicità** si riferisce alla lunghezza della sequenza generata prima che inizi a ripetersi. Per un LCG ben progettato, il periodo massimo è pari a m .

Tuttavia, anche con un periodo massimo, la qualità della sequenza **non è garantita automaticamente**. Le proprietà statistiche dell'LCG, come uniformità e indipendenza, sono fondamentali per applicazioni quali:

- simulazioni Monte Carlo
- modellazione del rischio
- analisi stocastiche

Scelte sbagliate dei parametri possono portare a diversi problemi:

- **Periodo breve**: se le condizioni non sono soddisfatte, la sequenza può ripetersi molto prima di m .
- **Correlazione tra i valori**: un LCG mal progettato può generare numeri fortemente correlati.

- **Distribuzione non uniforme:** i valori possono concentrarsi in certe regioni invece di coprire uniformemente l'intervallo.

Per valutare la qualità di un LCG si utilizzano test statistici come:

- test di Kolmogorov–Smirnov
- test di autocorrelazione

Questi test verificano se i numeri generati possiedono le caratteristiche desiderate di una sequenza casuale.

Implementazione di un Linear Congruential Generator in Python

Richiamando la relazione di ricorrenza dell'LCG descritta sopra, con i parametri moltiplicatore a , incremento c , modulo m e seed (X_0), possiamo implementare un LCG in modo diretto in Python.

(Il seguente codice è tratto da: <https://www.quantstart.com/articles/linear-congruential-generators-in-python/>)

```
import numpy as np

def lcg(seed, a, c, m, size):
    """
    Linear Congruential Generator (LCG) function.

    Parameters
    -----
    seed : `int`
        The initial value ( $X_0$ ) for the LCG.
    a : `int`
        The multiplier.
    c : `int`
        The increment.
    m : `int`
        The modulus.
    size : `int`
        The number of random numbers to generate.

    Returns
    -----
    `np.ndarray[float]`
        A NumPy array containing the generated sequence.
    """
    # Initialize the sequence array with zeros
    sequence = np.zeros(size)

    # Set the initial value
    sequence[0] = seed

    # Generate the sequence
    for i in range(1, size):
        sequence[i] = (a * sequence[i - 1] + c) % m

    return sequence
```

Nella funzione precedente:

- **seed:** il valore iniziale che dà avvio alla sequenza.
- **a:** il valore del moltiplicatore utilizzato nella formula dell'LCG.
- **c:** il valore dell'incremento aggiunto a ogni iterazione.
- **m:** il valore del modulo, che determina l'intervallo dei valori della sequenza.
- **size:** il numero di numeri pseudocasuali da generare nella sequenza.

La funzione inizializza un array NumPy, chiamato **sequence**, per memorizzare i valori generati. Imposta il primo valore dell'array uguale al **seed** e poi itera per la lunghezza desiderata della sequenza, calcolando ciascun valore successivo utilizzando la formula dell'LCG.

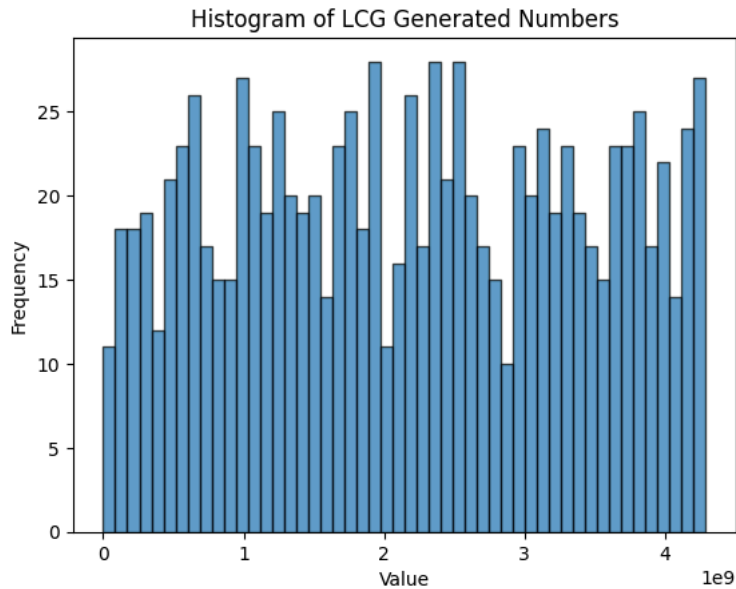
```
# Parameters for the LCG
seed = 42
a = 1664525
c = 1013904223
m = 2**32
size = 1000

# Generate the sequence
random_sequence = lcg(seed, a, c, m, size)

import matplotlib.pyplot as plt

# Plotting the histogram
plt.hist(random_sequence, bins=50, edgecolor='black', alpha=0.7)
```

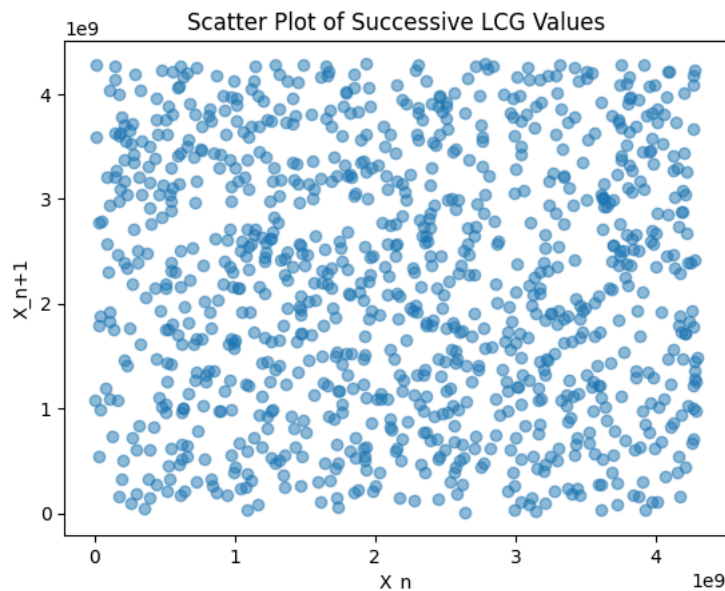
```
plt.title("Histogram of LCG Generated Numbers")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```



L'istogramma fornisce una rappresentazione visiva di come i valori generati sono distribuiti. Un LCG configurato correttamente dovrebbe mostrare una distribuzione abbastanza uniforme sull'intero intervallo di valori definito dal modulo m . Questa uniformità è fondamentale per applicazioni come le simulazioni Monte Carlo, dove la qualità dei numeri pseudocasuali può influenzare in modo significativo l'accuratezza dei risultati.

Per valutare ulteriormente l'indipendenza dei valori generati, è possibile creare uno scatter plot che rappresenti ciascun valore in funzione del suo successivo.

```
plt.scatter(random_sequence[:-1], random_sequence[1:], alpha=0.5)
plt.title("Scatter Plot of Successive LCG Values")
plt.xlabel("X_n")
plt.ylabel("X_{n+1}")
plt.show()
```



Nel **grafico di dispersione**, ogni punto rappresenta una coppia di valori successivi (X_n, X_{n+1}). Idealmente, i punti dovrebbero essere distribuiti in modo uniforme, senza formare alcun **pattern riconoscibile**.

Se i punti tendono a formare **cluster** o **linee**, ciò può indicare la presenza di **correlazione tra valori successivi**, suggerendo che i parametri dell'LCG necessitano di essere modificati per ottenere una migliore casualità.

Fai doppio clic (o premi Invio) per modificare

Esempio didattico di generatore di numeri pseudocasuali

A John von Neumann è attribuita la seguente citazione:

"Non c'è niente di più difficile che generare numeri casuali con un computer"

In questa sezione vedremo un esempio di generatore di numeri pseudocasuali.

Linear Congruential Generator (LCG)

Un LCG genera numeri secondo questa relazione ricorsiva:

$$X_{n+1} = (aX_n + c) \mod m$$

dove:

- X_n = numero pseudocasuale corrente
- a = moltiplicatore
- c = incremento
- m = modulo
- X_0 = seed iniziale (scelto dall'utente)

Quello qui presentato è il più semplice algoritmo per ottenere un numero pseudocasuale in modo uniforme.

Come funziona:

1. Si parte da un **seed** X_0 .
2. Si applica la formula:
 - si moltiplica per a
 - si aggiunge c
 - si fa il modulo m
3. Il risultato X_1 in modo iterativo viene usato per ottenere il valore successivo. X_1 diventa l'input per generare X_2
4. Ripetendo, si ottiene una sequenza di numeri "casuali".

Alcune osservazioni:

Un LCG può avere un **periodo massimo** pari a m (prima che la sequenza inizi a ripetersi), ma *solo* se soddisfa tre condizioni:

1. c è coprimo con m
2. $a - 1$ è multiplo di tutti i fattori primi di m
3. Se m è multiplo di 4, allora anche $a - 1$ deve esserlo

In rete si trova che molte librerie "storiche" usavano, per esempio, come valori:

- $m = 2^{31}$
- $a = 1103515245$
- $c = 12345$

Esempio di un LCG *implementato da zero* in Python

Ecco un **codice minimale**, senza usare alcuna libreria, che implementa un LCG vero e funzionante:

```
# Linear Congruential Generator (LCG)

# Parametri classici
m = 2**31
```